

Sujet d'examen UE NFE204 - Session 1

Année universitaire 2024-2025

Examen première session : 28 janvier 2025

Consignes

Durée de l'examen : 3h00; Calculatrice autorisée ; Documents autorisés : 5 pages recto-verso de note personnelles. Ce sujet contient 4 pages.

Nous gérons un site de commerce électronique qui recense des restaurants et permet à des internautes de les noter. Au départ, nous utilisons un système relationnel, dont voici le schéma (très simplifié).

- Restaurant (**id**, nom, catégorie)
- Internaute (**id**, nom)
- Notation (**idRestau**, **idInternaute**, note, commentaire)

La catégorie du restaurant indique son niveau de prix, entre * et ****. Voici un tout petit échantillon de la base.

id	nom	catégorie	id	nom	idRestau	idInternaute	note	commentaire
20	Bobo	**	100	Marie	20	100	1	Nul
21	Le régal	****	101	Adrien	20	101	5	Parfait
					21	101	3	Moyen

La table *Restaurant* La table *Internaute* La table *Notation*

On veut implanter un algorithme de filtrage collaboratif. Un des calculs consiste notamment à déterminer la note moyenne, en fonction de certains critères comme la catégorie du restaurant ou l'internaute.

1 Documents structurés (4 pts)

1. Donnez une représentation de l'échantillon ci-dessus sous la forme d'une collection de documents JSON, centrée sur les entités "Restaurant". Nous l'appellerons représentation A.
2. Proposez une autre représentation, centrée sur les entités "Internaute". Nous l'appellerons représentation B.

Solution : Vu de nombreuses fois en cours. Voici pour la représentation A.

```
[{
  "_id": 10,
  "nom": "Bobo",
  "catégorie": "**",
  "notes": [{ "id_inter": 100, "nom": "Marie",
               "commentaire": "Nul", "note": 1 },
             { "id_inter": 101, "nom": "Adrien",
               "commentaire": "Parfait", "note": 5 }
  ],
},
{
  "_id": 21,
  "nom": "Le régal",
  "catégorie": "****",
  "notes": [{ "id_inter": 101, "nom": "Adrien",
               "commentaire": "Moyen", "note": 3 }
  ]
}]
```

2 Modélisation Cassandra (5 pts)

On veut proposer un schéma Cassandra pour explorer cette base. Rappelons d'abord ce qu'est une clé composite en Cassandra : *A compound primary key consists of the partition key and the clustering key. The partition key determines which node stores the data. Rows for a partition key are stored in order based on the clustering key.*

On crée les tables suivantes, correspondant à une modélisation Cassandra de nos documents de type A :

```
create table Restaurant (catégorie text, id_restau text, nom restau text, adresse text,  
                        primary key (catégorie, id_restau));
```

```
create type Internaute (id_internaute integer, nom text, prénom text);
```

```
create table Notation (id_restau text,  
                      internaute frozen<Internaute>,  
                      note integer,  
                      commentaire text,  
                      primary key (id_restau, internaute));
```

1. Commentez chacune des affirmations suivantes, indiquez si elles sont vraies ou fausses et pourquoi.
 - (a) Les lignes de Restaurant et les lignes de Notation sont sur le même serveur
 - (b) Les restaurants d'une même catégorie sont sur le même serveur
 - (c) Le choix de catégorie comme clé de partitionnement est judicieux.
 - (d) Les notations d'un restaurant sont sur un seul serveur mais distribuées de manière aléatoire dans le stockage local.
 - (e) Les notations d'un même internaute sont sur un seul serveur et triées sur l'identifiant du restaurant.
2. Expliquez maintenant quelles sont les requêtes et opérations nécessaires pour calculer la moyenne des notes d'une catégorie de restaurant (par exemple les '***'). Discutez de l'efficacité.
3. Proposez un second schéma avec une table Internaute correspondant à nos documents de type B (première question du sujet).

Solution :

1.
 - (a) Non puisqu'elles n'ont pas la même clé de partitionnement.
 - (b) Oui, la catégorie est la clé de partitionnement
 - (c) Non car il n'y a que 5 catégories, donc un partitionnement limité à 5 fragments.
 - (d) Oui sur un seul serveur mais de plus le stockage est contigu.
 - (e) Pas du tout car l'internaute n'est pas clé de partitionnement.
2. Une première requête va récupérer tous les restaurants '***'. C'est un parcours séquentiel sur un seul serveur.

```
select * from Restaurant where catégorie='***';
```

Ensuite, pour chaque identifiant de restaurant on récupère les notes. Là aussi, parcours séquentiel sur un seul serveur (mais autant de requêtes que de restaurants).

```
select note from Notation where id_restau=<id>;
```

On a toutes les notes, on peut calculer la moyenne au fur et à mesure.

3. On peut tout mettre dans une seule table, en intégrant les notes sous forme de liste (en espérant qu'il n'y en aura pas trop).

```
create type Note (id_restau integer, note integer, commentaire text);
```

```
create table Internaute (id_internaute text,  
                        nom text,  
                        notes list(Note),  
                        primary key (id_internaute));
```

3 Spécification de calculs distribués (4 pts)

On suppose que les informations sont disponibles sous la forme d'une collection de documents JSON structurés selon la représentation A (cf. première question du sujet).

1. Indiquez le traitement MapReduce qui, appliqué à cette collection de documents, permet de produire les lignes à insérer dans la table *Cassandra Notation* donné dans la section précédente.
2. Indiquez le traitement MapReduce qui, appliqué à cette collection de documents, permet de produire les lignes à insérer dans la table *Cassandra Internaute* dont vous avez proposé un schéma dans la section précédente. Vous pouvez aussi expliquer comment ce traitement permet de produire la représentation B à partir de la représentation A.
3. La phase de Reduce (et donc celle de *shuffle*) est-elle nécessaire dans les deux cas ? Expliquer.

Pour spécifier les calculs MapReduce, donner les fonctions de Map et de Reduce, en javascript, en pseudo-code, au pire en langage naturel en étant le plus précis possible.

Solution :

1. Pour la fonction de Map, on reçoit un document représentant un restaurant. Il faut boucler sur les notes, et émettre une paire clé-valeur avec comme clé l'identifiant du restaurant et l'internaute. On peut en fait directement insérer dans la table *Notation* dès le Map puisqu'aucun regroupement n'est nécessaire.
2. Pour le second en revanche, le Reduce est indispensable. Le Map émet, dans une boucle sur les notes, des paires dont la clé est l'identifiant de l'internaute. Le *shuffle* regroupe les paires de même clé ce qui permet de constituer la liste des notes, et le Reduce peut alors insérer une ligne pour chaque groupe dans la table *Internaute*.

4 Recherche d'information (4 pts)

On ajoute dans les documents des commentaires sur les restaurants, désignés par $c_1, \dots, c_5$. On veut maintenant créer un index plein texte pour rechercher des commentaires. On va travailler sur l'échantillon suivant.

- c_1 Plats médiocres, bonne carte des vins, accueil cordial.
- c_2 Le vin est moyen, les desserts sont mauvais, mais le décor est agréable.
- c_3 En général, le vin est trop cher au restaurant, mais ici la carte des vins est complète et de qualité.
- c_4 Passons sur le décor d'un goût douteux. Les plats et desserts sont délicieux, c'est tout ce qui compte après tout.
- c_5 Plat, dessert et un verre de vin pour un prix correct. Le plat du jour est toujours intéressant.

On va se limiter au vocabulaire (vin, plat, décor, dessert). (NB : on suppose une phase préalable de normalisation qui élimine les pluriels, majuscules, etc.)

1. Donnez une matrice d'incidence contenant les tf, et un tableau donnant les idf (sans le log), pour les termes précédents. Placez les termes en ligne, les commentaires en colonne.

Solution :

	c1	c2	c3	c4	c5
vin 5/4	1	1	2	0	1
plat 5/3	1	0	0	1	2
décor 5/2	0	1	0	1	0
dessert 5/3	0	1	0	1	1

2. Donnez les normes des vecteurs représentant ces commentaires.

Solution : Les normes

$$\|c_1\| = \sqrt{1+1}; \|c_2\| = \sqrt{3}; \|c_3\| = \sqrt{4}; \|c_4\| = \sqrt{3}, \|c_5\| = \sqrt{6}$$

3. Donner les résultats classés par similarité cosinus basée sur les tf (on ignore l'idf) pour la requête associant les deux termes *décor* et *dessert*. Expliquez brièvement la raison du classement pour le premier document.

Solution : Les cosinus (requête non normalisée).

- c2 : $\frac{1+1}{\sqrt{3}}$
- c4 : $\frac{1+1}{\sqrt{3}}$
- c5 : $\frac{1}{\sqrt{6}}$

4. Pour les documents c2 et c4, qu'est-ce qui change si on ne met pas de restriction sur le vocabulaire (tous les mots sont indexés) ?

Solution : Dans ce cas le plus petit document (c2) l'emporte car sa norme est plus petite. Intuitivement : il parle plus *spécifiquement* de décor et de dessert que c4.

5 Systèmes distribués (3 pts)

Rappelons que Elastic Search partitionne les documents avec une méthode de hachage statique, alors que Cassandra s'appuie sur la technique du *consistent hashing*.

On suppose que notre grappe de serveur comprend n serveurs.

1. Expliquez comment un document de notre collection A de documents JSON est inséré dans une grappe ElasticSearch (donnez l'algorithme déterminant le serveur).
2. Même question pour une ligne de la table Cassandra *Notation*.
3. Notre collection double de taille en quelques jours. Que peut-on faire pour assurer la scalabilité avec chacune des deux solutions envisagées, ElasticSearch et Cassandra ?